ТУСУР
TUSUR UNIVERSITY

Подходы к детектированию объектов на изображениях. Области применения

ТУСУР
TUSUR UNIVERSITY

Исторические аспекты детектирования объектов на изображении

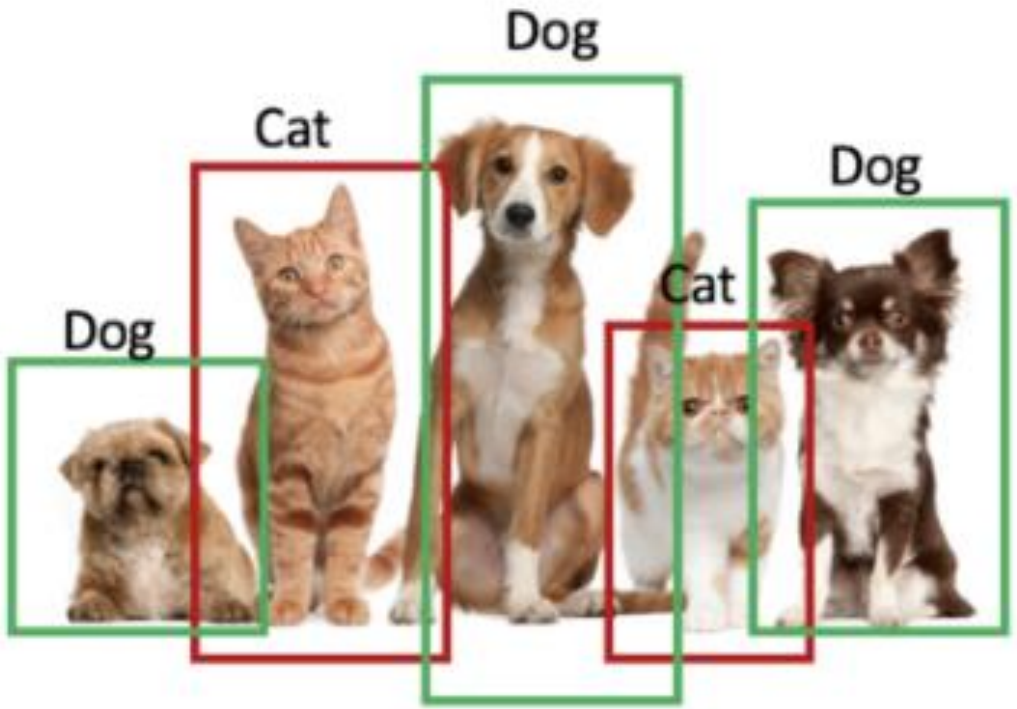
История

Детектирование объектов – задача определения наличия объекта на представленном изображении.

Ещё несколько лет назад, задача детектирования являлась чрезвычайно трудной, однако, на данный момент является не только возможной, но активно изучается такими компаниями как Google и IBM.

Всю историю детектирования объектов можно разделить на два больших этапа:

- Традиционный этап
- Этап глубокого обучения



Традиционный этап

Детектор Виолы-Джонса, разработанный в 2001 году Паулом Виола и Майклом Джонсом, позволяет в реальном времени обнаруживать лица.

Детектор HOG был представлен в 2005 году Н. Далалем и Б. Триггсем. Он основывается на «блоках», схожих со скользящими окнами, и широко известен за распознавание пешеходов.

Модель DPM (deformable Part-based Model), представленная в 2008 году представляет собой расширение HOG, в котором объекты рассматриваются по отдельным частям (колёса, рама, педали велосипеда).



Этап глубокого обучения

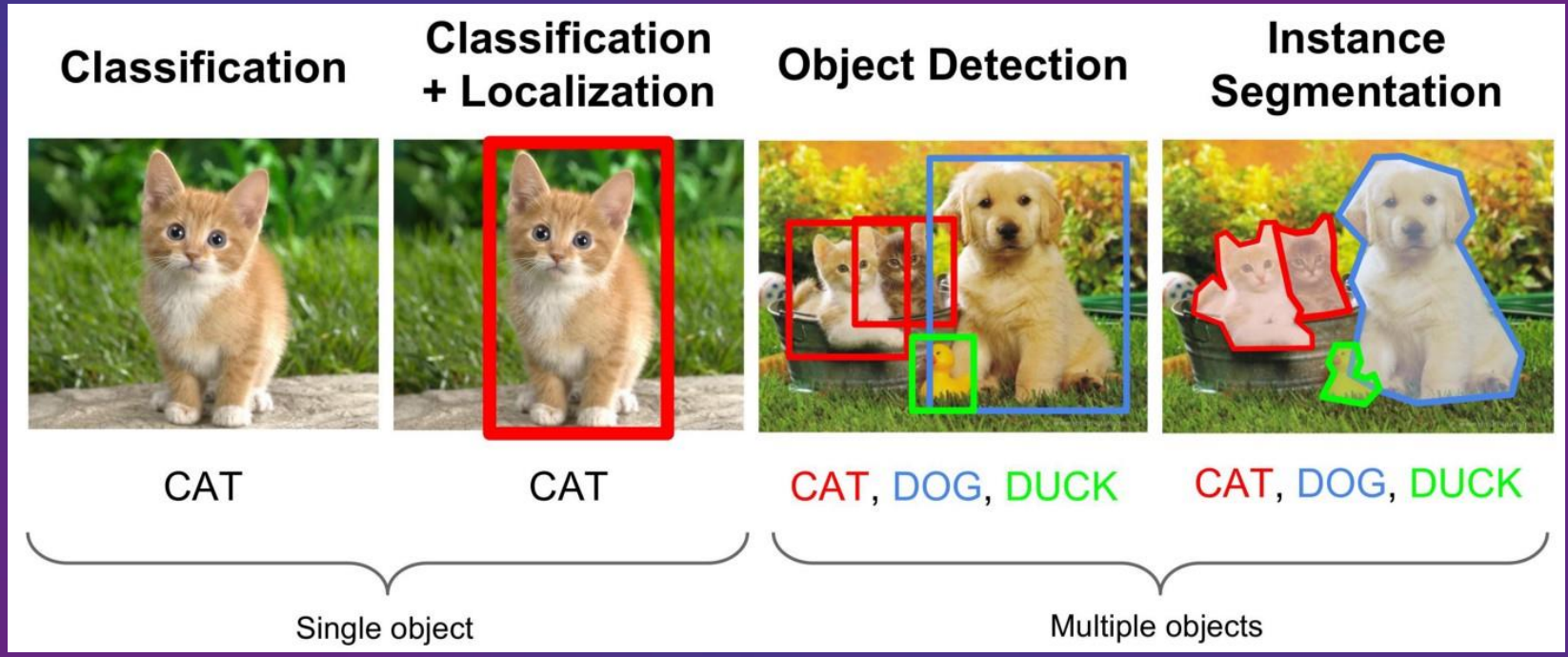
ТУСУР
TUSUR UNIVERSITY

К 2010 году задача детектирования объектов достигла своего плато, из-за пределов производительности ручных функций, но уже в 2012 году стали сильнее изучаться свёрточные нейронные сети, а именно сети глубокого обучения, благодаря чему удалось перейти на новый уровень точности в обнаружении объектов.

Так в 2014 года был представлен метод R-CNN, базирующийся на двух этапах: селективный поиск возможных регионов объекта и классификация признаков полученных регионов. В дальнейшем данный метод был не раз переработан и улучшен, благодаря чему появились такие методы как Fast R-CNN, Faster R-CNN, Mask R-CNN.

В 2016 году был представлен одноэтапный алгоритм YOLO (You Only Look Once), отличающийся от R-CNN тем, что анализирует изображение целиком, разделяя его на сетку и проверяя наличие центра объекта в каждой из ячеек.

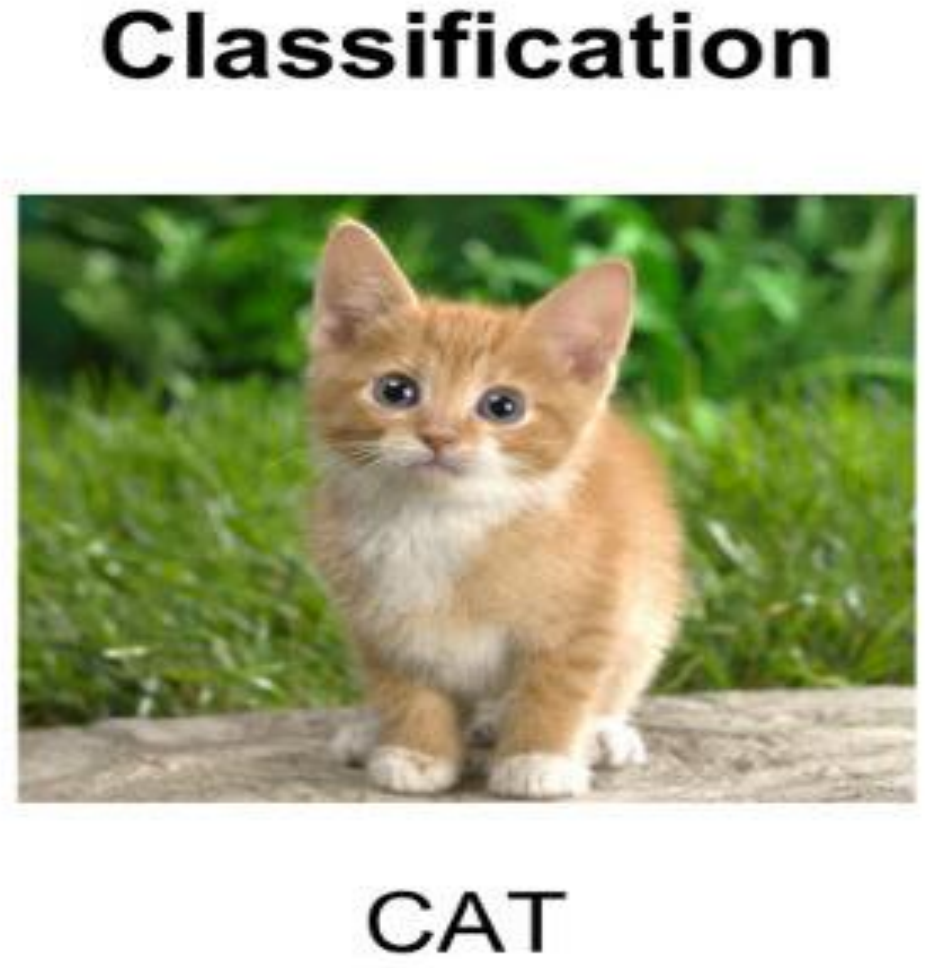
ТУСУР
TUSUR UNIVERSITY



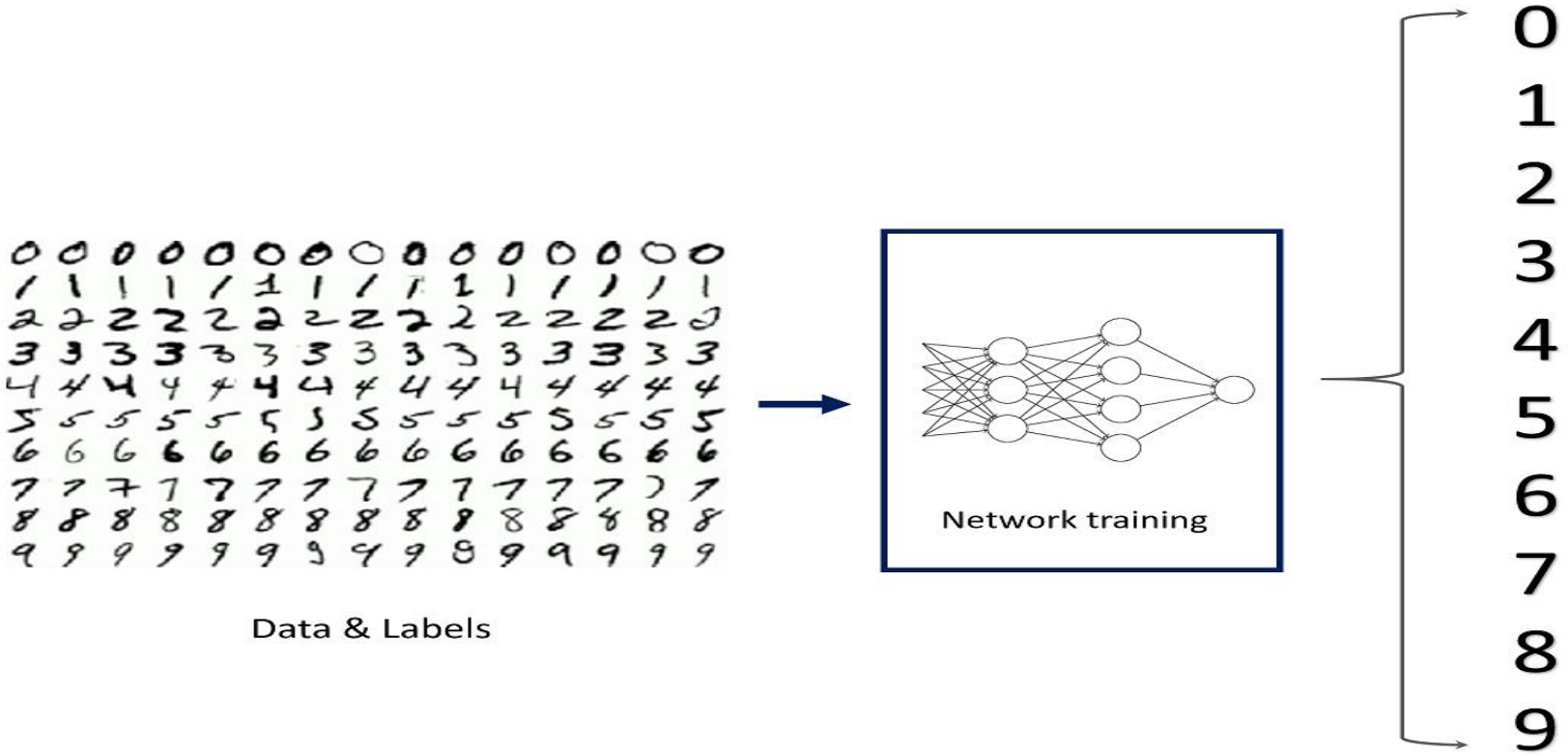
Основные задачи детектирования объектов на изображениях

Задача классификации

Классификация изображений — это фундаментальная задача, которая пытается понять все изображение как единое целое. Цель состоит в том, чтобы классифицировать изображение, назначив ему определенную метку. Как правило, классификация изображений относится к изображениям, на которых появляется и анализируется только один объект.



Задача классификации



Data & Labels

Загрузка и нормализация данных

ТУСУР
TUSUR UNIVERSITY

```
from numpy import mean
from numpy import std
from matplotlib import pyplot as plt
from sklearn.model_selection import KFold
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Flatten
from tensorflow.keras.optimizers import SGD
```

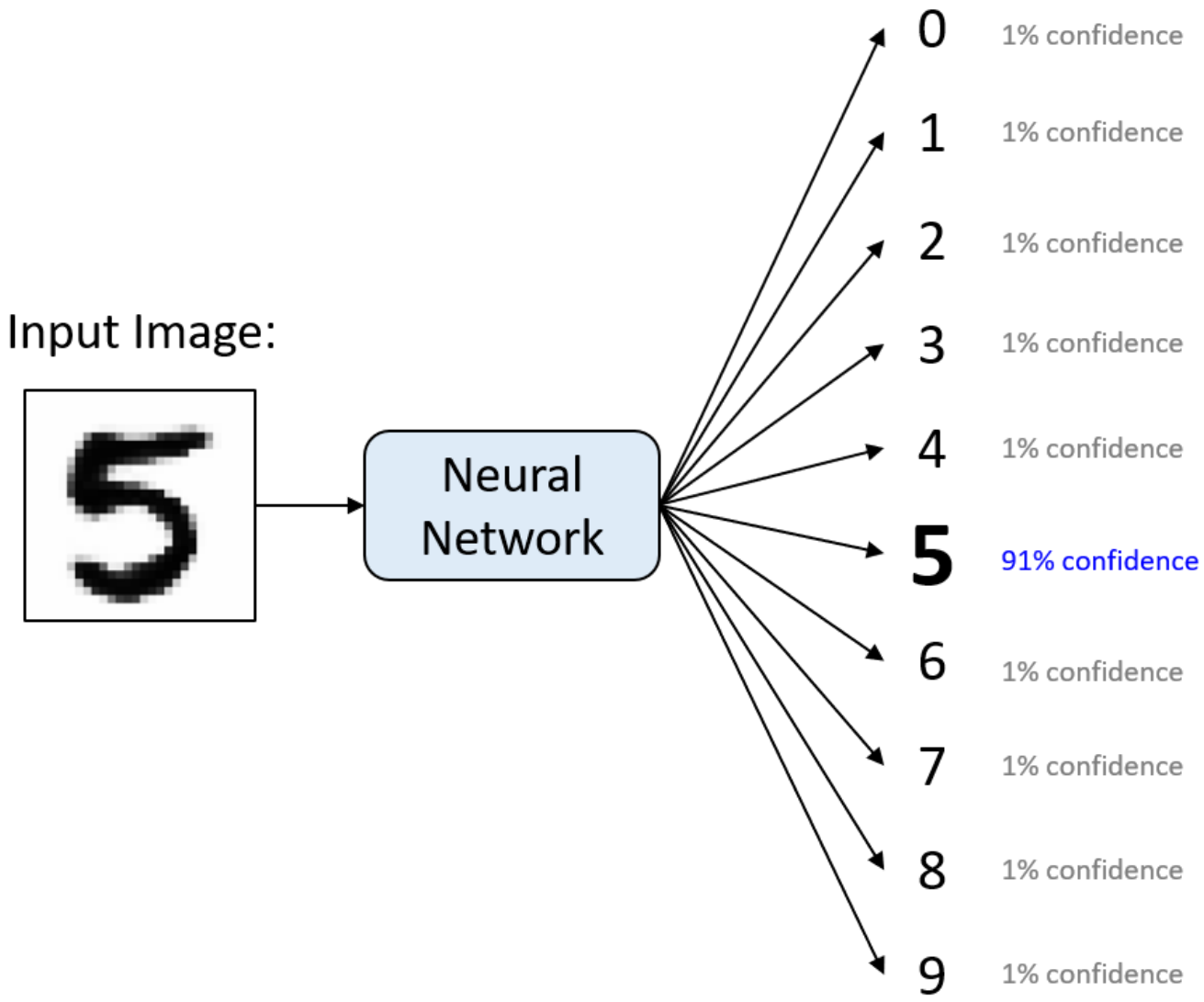
```
# load dataset
(trainX, trainY), (testX, testY) = mnist.load_data()
# reshape dataset to have a single channel
trainX = trainX.reshape((trainX.shape[0], 28, 28, 1))
testX = testX.reshape((testX.shape[0], 28, 28, 1))
# one hot encode target values
trainY = to_categorical(trainY)
testY = to_categorical(testY)
# convert from integers to floats
train_norm = train.astype('float32')
test_norm = test.astype('float32')
# normalize to range 0-1
train_norm = train_norm / 255.0
test_norm = test_norm / 255.0
```

Обучение модели

ТУСУР
TUSUR UNIVERSITY

```
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
model.add(Dense(10, activation='softmax'))
# compile model
opt = SGD(learning_rate=0.01, momentum=0.9)
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
# fit model
history = model.fit(trainX, trainY, epochs=10, batch_size=32, validation_data=(testX, testY), verbose=0)
# evaluate model
_, acc = model.evaluate(testX, testY, verbose=0)
# stores scores
scores.append(acc)
histories.append(history)
```

Задача классификации



https://machinelearningmastery.com/how-to-develop-a-convolutional-neural-network-from-scratch-for-mnist-handwritten-digit-classification/

Задача локализации

Локализация объекта – это фундаментальная задача, целью которой является определение местоположения цели на изображении или видео.

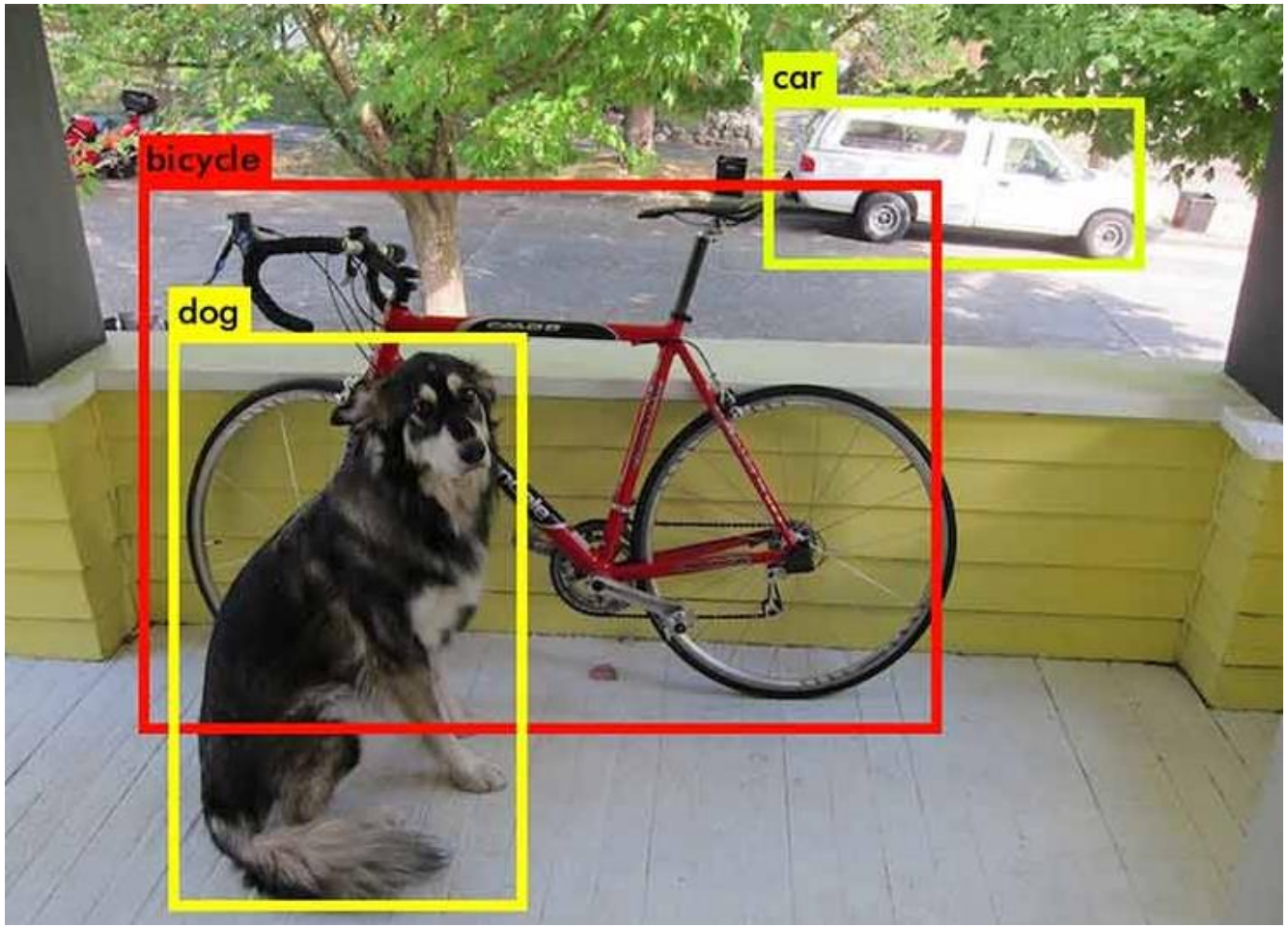
Как правило, сначала определяется центр объекта, а затем определяются ширина и высота объекта. Другим вариантом, может быть определение нижнего левого края объекта, и ширины с высотой.



Детектирование объектов

ТУСУР
TUSUR UNIVERSITY

Обнаружение объектов – это определение местоположения и классификация объектов. Можно сказать, что это использование классификации изображений и локализации объекта одновременно для множества объектов.



```
def feature_extractor(inputs):
    x = ks.layers.Conv2D(16, activation='relu', kernel_size=3, input_shape=(75, 75, 1))(inputs)
    x = ks.layers.AveragePooling2D((2, 2))(x)
    x = ks.layers.Conv2D(32, activation='relu', kernel_size=3)(x)
    x = ks.layers.AveragePooling2D((2, 2))(x)
    x = ks.layers.Conv2D(64, activation='relu', kernel_size=3)(x)
    x = ks.layers.AveragePooling2D((2, 2))(x)
    return x

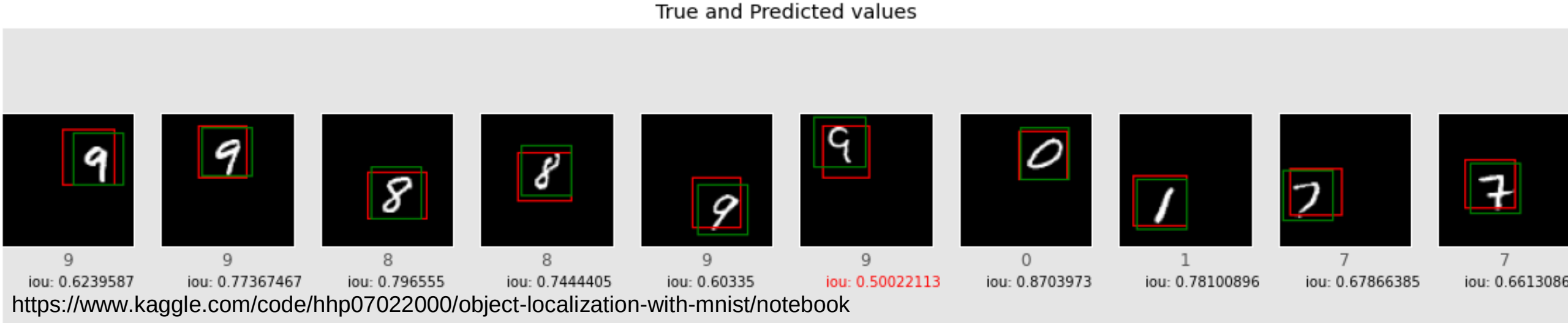
def dense_layer(inputs):
    x = ks.layers.Flatten()(inputs)
    x = ks.layers.Dense(128, activation='relu')(x)
    return x
```

```
def classifier(inputs):
    classifier_output = ks.layers.Dense(10, activation='softmax',name='classifier')(inputs)
    return classifier_output

def bounding_box_regression(inputs):
    bounding_box_regression_output = ks.layers.Dense(units='4', name='bounding_box')(inputs)
    return bounding_box_regression_output

def final_model(inputs):
    feature_cnn = feature_extractor(inputs)
    dense_output = dense_layer(feature_cnn)
    classification_output = classifier(dense_output)
    bounding_box = bounding_box_regression(dense_output)
    model = ks.Model(inputs=inputs, outputs=[classification_output, bounding_box])
    return model
```

TYCYP
TUSUR UNIVERSITY

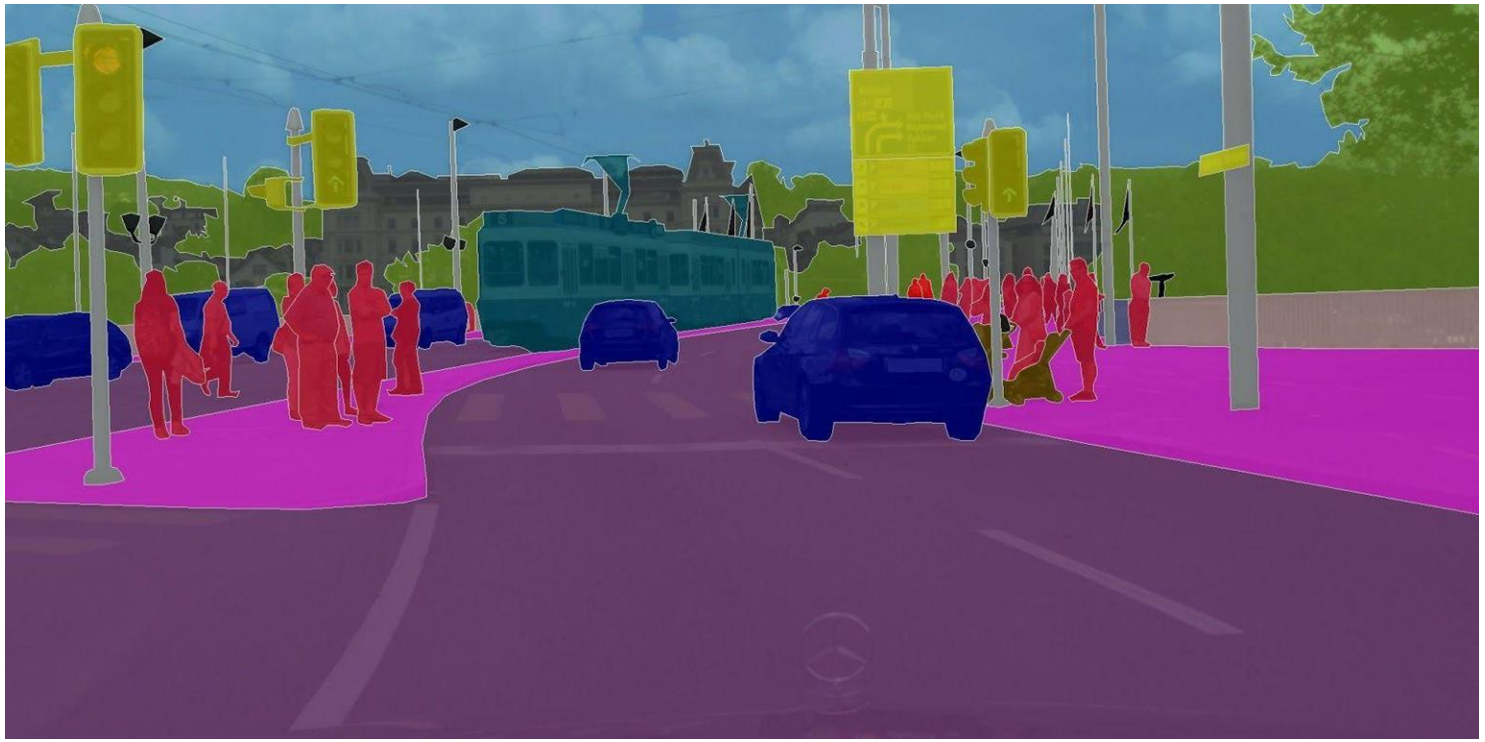


https://www.kaggle.com/code/hhp07022000/object-localization-with-mnist/notebook

Семантическая сегментация

ТУСУР
TUSUR UNIVERSITY

Семантическая сегментация — это процесс разделения цифрового изображения на несколько сегментов. Цель сегментации заключается в упрощении и/или изменении представления изображения, чтобы его было проще и легче анализировать.



Семантическая сегментация

```
epochs = 20

if kaggle_commit:

    epochs = 5

from keras_segmentation.models.unet import vgg_unet
n_classes = 23

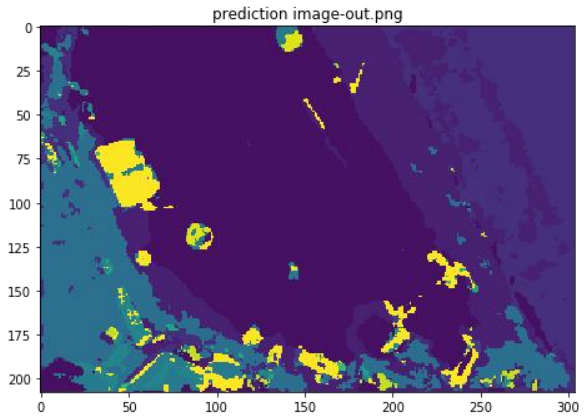
model = vgg_unet(n_classes=n_classes ,  input_height=416, input_width=608  )
model.train(

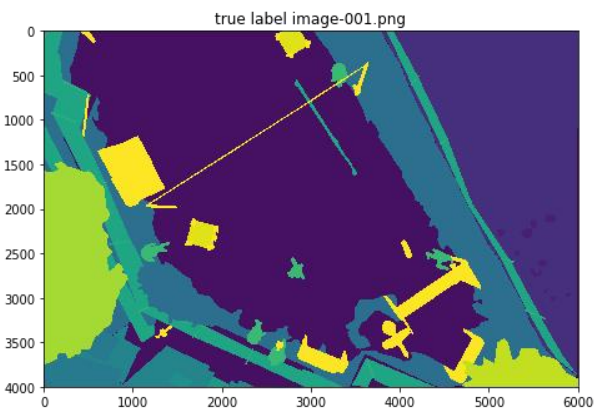
    train_images =  "/content/original_images/",

    train_annotations = "/content/label_images_semantic/",

    checkpoints_path = "vgg_unet" , epochs=epochs)
```







https://www.kaggle.com/code/bulentsiyah/deep-learning-based-semantic-segmentation-keras

Сегментация экземпляров

Сегментация экземпляров — это метод обнаружения, сегментации и классификации каждого отдельного объекта на изображении. Мы можем ссылаться на сегментацию экземпляров как на комбинацию семантической сегментации и обнаружения объектов (обнаружение всех экземпляров категории на изображении) с дополнительной функцией разграничения отдельных экземпляров любого конкретного класса сегментов, добавленной к стандартной задаче сегментации.



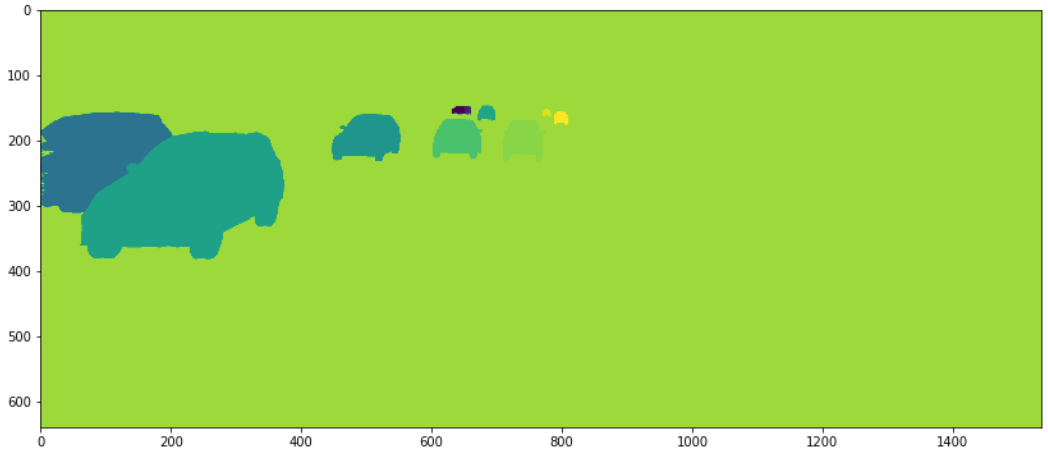
```
class UNet(nn.Module):
    def __init__(self, n_classes):
        super(UNet, self).__init__()
        self.dc0 = double_conv(3, 32)
        self.mp0 = nn.MaxPool2d(2)
        self.dc1 = double_conv(32, 64)
        self.mp1 = nn.MaxPool2d(2)
        self.dc2 = double_conv(64, 128)
        self.mp2 = nn.MaxPool2d(2)
        self.dc3 = double_conv(128, 256)
        self.mp3 = nn.MaxPool2d(2)
        self.dc4 = double_conv(256, 256)
        self.up0 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.dc5 = double_conv(512, 128)
        self.up1 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.dc6 = double_conv(256, 64)
        self.up2 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.dc7 = double_conv(128, 32)
        self.up3 = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
        self.last = nn.Conv2d(64, n_classes, 1)
```

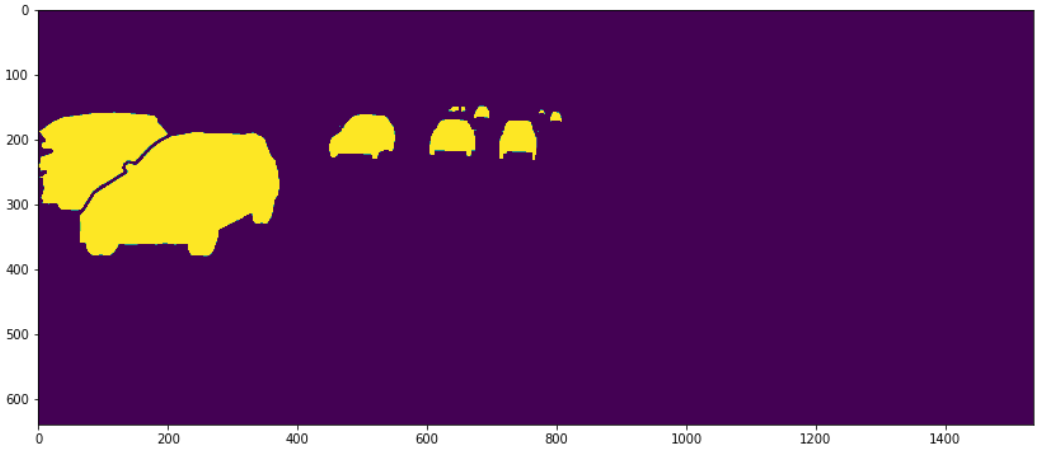
```
    def forward(self, x):
        x0 = self.dc0(x)
        x1 = self.dc1(self.mp0(x0))
        x2 = self.dc2(self.mp1(x1))
        x3 = self.dc3(self.mp2(x2))
        x = self.dc4(self.mp3(x3))
        x = torch.cat([x3, self.up0(x)],1)
        x = self.up1(self.dc5(x))
        x = torch.cat([x2, x],1)
        x = self.up2(self.dc6(x))
        x = torch.cat([x1, x],1)
        x = self.up3(self.dc7(x))
        x = torch.cat([x0, x],1)
        x = self.last(x)
        return x

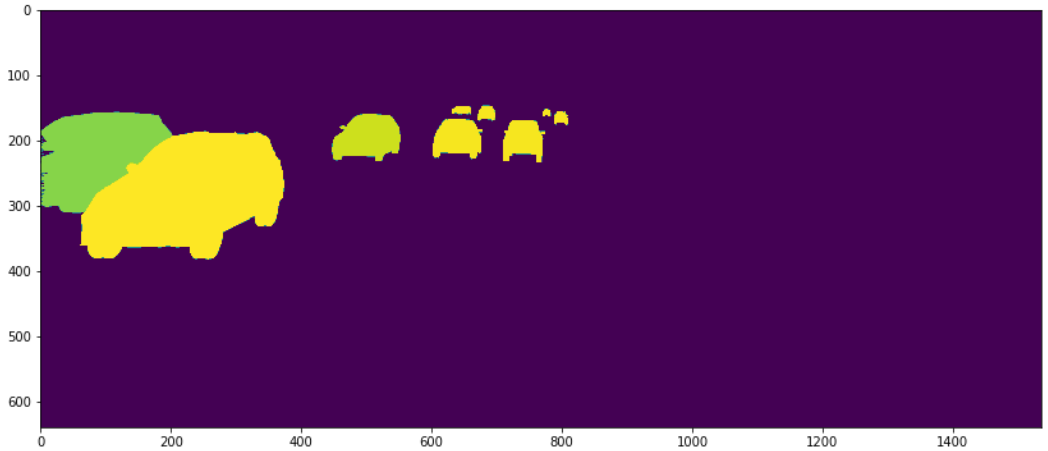
model = UNet(8).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.001)
for epoch in range(n_epochs):
    torch.cuda.empty_cache()
    gc.collect()
    train_model(epoch)
    dev_loss = evaluate_model(epoch)
    if epoch == 0:
        best_loss = dev_loss
    else:
        if dev_loss <= best_loss:
            best_loss = dev_loss
            torch.save(model.state_dict(), './is_unet_v2.pth')
```

TYCYP
TUSUR UNIVERSITY









https://www.kaggle.com/code/ryunosukeishizaki/instance-segmentation-pipeline

ТУСУР
TUSUR UNIVERSITY

Методы детектирования объектов на изображениях

Традиционные методы

ТУСУР
TUSUR UNIVERSITY

Традиционные методы базируются на использовании алгоритмов, не связанных с глубоким обучением – метод Виолы-Джонса, гистограмм ориентированных методов, деформации частичной модели и др.

Базовый алгоритм работы традиционных методов:

- Предобработка изображения;
- Извлечение признаков (HAAR, HOG, SURF и т.д.);
- Классификация объекта на основе выделенных признаков.

Хоть в последние годы всё чаще используются методы глубокого обучения, традиционные методы до сих пор используются в некоторых приложениях для распознавания объектов, особенно в мобильных устройствах, где требуются решения с малыми требованиями к вычислительными мощностями.

Масштабно-инвариантное преобразование признаков (SIFT)

ТУСУР
TUSUR UNIVERSITY

SIFT - это алгоритм компьютерного зрения для обнаружения, описания и сопоставления локальных особенностей изображений, изобретенный Дэвидом Лоу в 1999 году.

Метод Лоу для генерации признаков изображения преобразует изображение в большой набор векторов признаков, каждый из которых инвариантен к перемещению, масштабированию и вращению изображения, частично инвариантен к изменениям освещения и устойчив к локальным геометрическим искажениям.

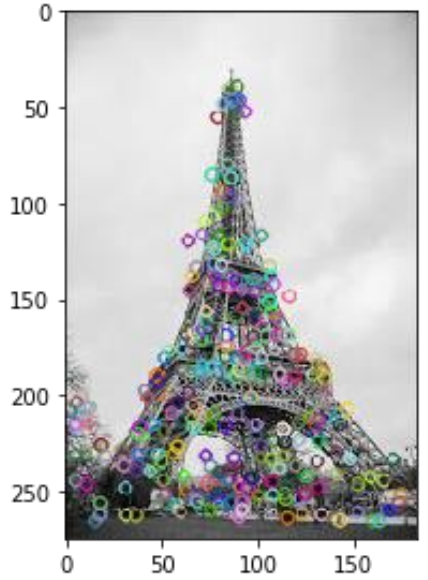
Алгоритм работы:

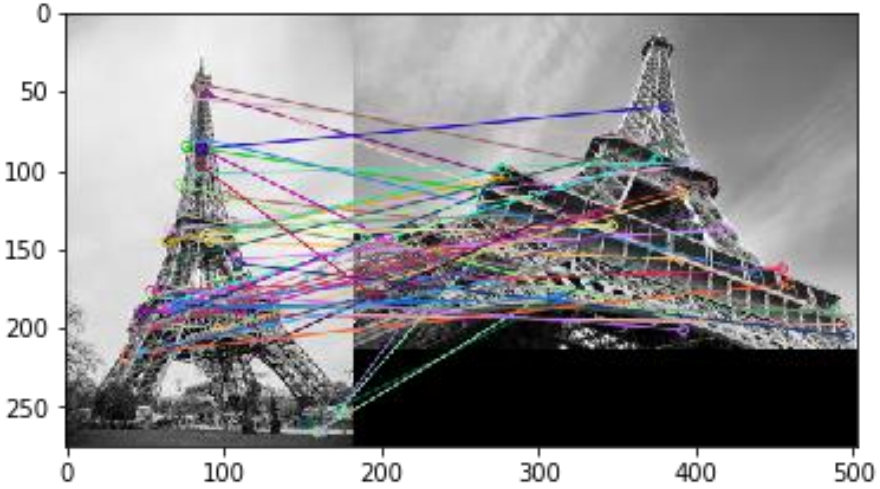
- Выявление и локализация ключевых точек
- Назначение ориентаций для ключевых точек
- Дескриптор ключевых точек

Масштабно-инвариантное преобразование признаков (SIFT)

ТУСУР
TUSUR UNIVERSITY

```
import cv2
import matplotlib.pyplot as plt
%matplotlib inline
img1 = cv2.imread('eiffel_2.jpeg')
img2 = cv2.imread('eiffel_1.jpg')
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
sift = cv2.xfeatures2d.SIFT_create()
keypoints_1, descriptors_1 = sift.detectAndCompute(img1,None)
keypoints_2, descriptors_2 = sift.detectAndCompute(img2,None)
bf = cv2.BFMatcher(cv2.NORM_L1, crossCheck=True)
matches = bf.match(descriptors_1,descriptors_2)
matches = sorted(matches, key = lambda x:x.distance)
img3 = cv2.drawMatches(img1, keypoints_1, img2, keypoints_2, matches[:50], img2, flags=2)
```





https://www.analyticsvidhya.com/blog/2019/10/detailed-guide-powerful-sift-technique-image-matching-python/

Метод Виолы-Джонса

ТУСУР
TUSUR UNIVERSITY

Метод Виолы — алгоритм, позволяющий обнаруживать объекты на изображениях в реальном времени, предложенный Паулом Виола и Майклом Джонсом в 2001 году, который основывается на суммировании пикселей в прямоугольных областях.

Основные принципы, на которых основан метод, таковы:

- используются изображения в интегральном представлении, что позволяет вычислять быстро необходимые объекты;
- используются признаки Хаара, с помощью которых происходит поиск нужного объекта (в данном контексте, лица и его черт);
- используется бустинг для выбора наиболее подходящих признаков для искомого объекта на данной части изображения;
- все признаки поступают на вход классификатора, который даёт результат «верно» либо «ложь»;
- используются каскады признаков для быстрого отбрасывания окон, где не найдено лицо.

Признаки Хаара

Признаки Хаара – признаки цифрового изображения, используемые в распознавании образов, основанные на вейвлетах Хаара и состоящие из смежных прямоугольных областей, которые позиционируются на изображении.

Вейвлет Хаара – математическая функция для анализа компонентов данных, основанная на ортогональной системе функций.

Основным принципом признаков Хаара является суммирование интенсивностей в прямоугольных областях и вычисление разности между суммами.

Виды признаков Хаара:

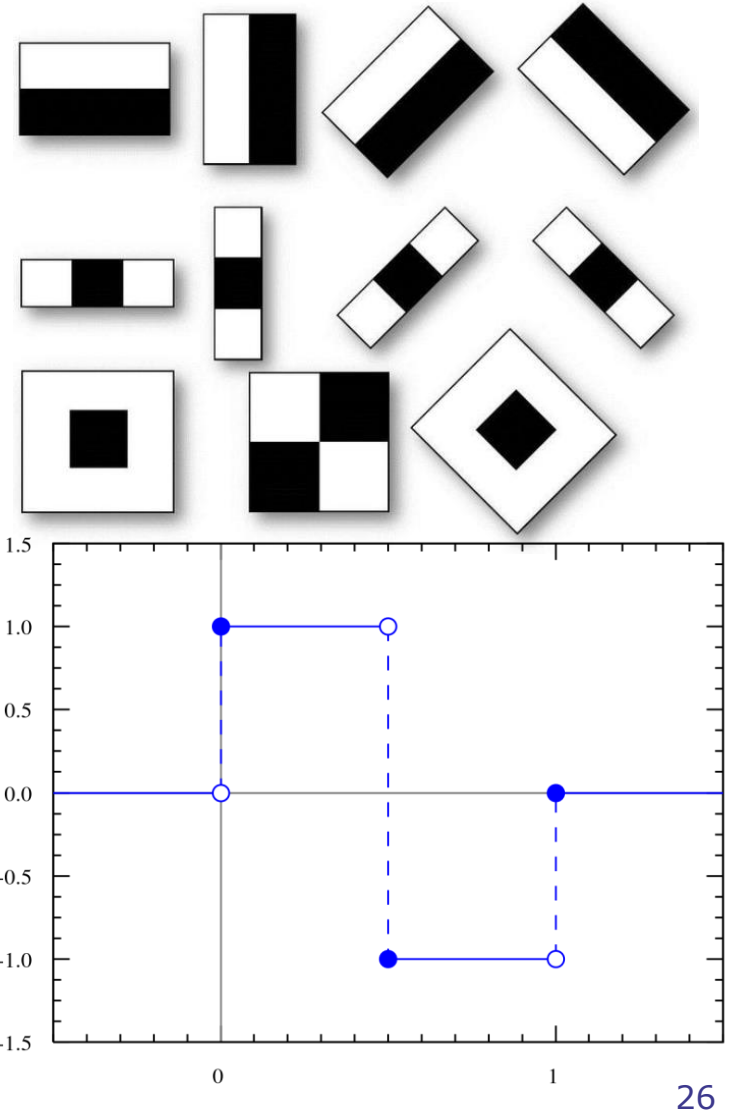
- прямоугольные признаки – могут быть как 2-прямоугольными, так и 3- и 4-прямоугольными;
- наклонные признаки – разновидность признаков Хаара, однако, наклонённые на 45 градусов.

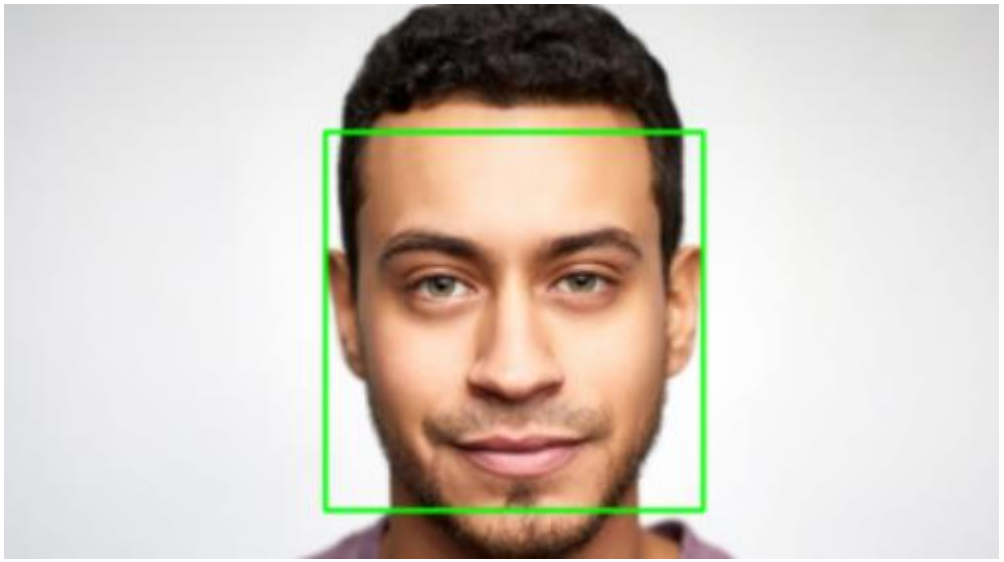
$$\psi(\mathrm{x}) = \begin{cases} 1, 0 \le x < 0{,}5 \\ -1.\, 0{,}5 \le x < 1 \\ 0, x \notin [0; 1) \end{cases}$$

$$\int_{-\infty}^{\infty} \psi(x)dx = 0$$

$$\varphi(x) = \begin{cases} 1, 0 \le x < 1 \\ 0, x \notin [0; 1) \end{cases}$$

$$\int_{-\infty}^{\infty} \varphi(x)dx = 1$$





```
import cv2
import numpy as np
import matplotlib.pyplot as plt
original_image = cv2.imread("1.jpg")
grayscale_image = cv2.cvtColor(original_image, cv.COLOR_BGR2GRAY)
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
detected_faces = face_cascade.detectMultiScale(grayscale_image)
for (column, row, width, height) in detected_faces:
    cv2.rectangle(original_image,(column, row),(column + width, row + heig
ht),(0, 255, 0),4)
```


https://towardsdatascience.com/viola-jones-algorithm-and-haar-cascade-classifier-ee3bfb19f7d8

Гистограмма ориентированных градиентов (HOG)

ТУСУР
TUSUR UNIVERSITY

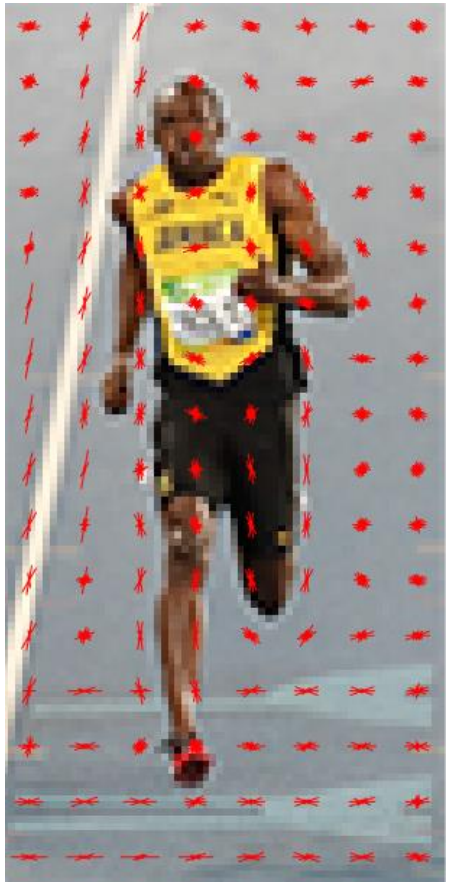
Дескриптор признаков HOG – представление изображения в виде вектора признаков, которыми являются гистограммы направлений градиентов.

Алгоритм получения гистограммы ориентированных векторов:

- Предобработка изображения
- Расчёт градиентов изображения
- Вычисление гистограммы градиентов для ячеек 8x8
- Нормализация через блоки 16x16
- Расчёт итогового вектора на основе нормализованных гистограмм

Для визуализации достаточно построить нормализированные гистограммы, которые будут описывать форму объекта.

Градиент — вектор, своим направлением указывающий направление возрастания некоторой скалярной величины, а по величине равный скорости роста этой величины в этом направлении.



```
import matplotlib.pyplot as plt

from skimage.feature import hog

from skimage import data, exposure

image = data.astronaut()

fd, hog_image = hog(image, orientations=8,
pixels_per_cell=(16, 16), cells_per_block=(1, 1),
visualize=True, channel_axis=-1)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(8, 4),
sharex=True, sharey=True)

ax1.axis('off')

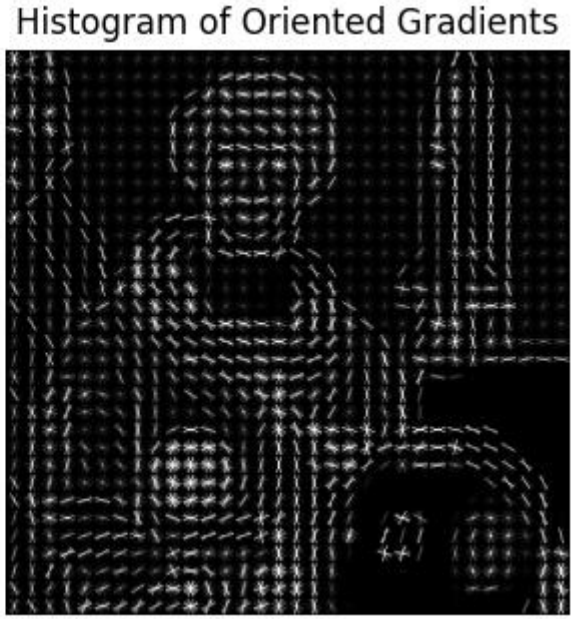
ax1.imshow(image, cmap=plt.cm.gray)

ax1.set_title('Input image')

# Rescale histogram for better display

hog_image_rescaled =
exposure.rescale_intensity(hog_image, in_range=(0, 10))
```





https://scikit-image.org/docs/dev/auto_examples/features_detection/plot_hog.html

Методы глубокого обучения

Нейронные методы детектирования объектов на изображениях заключаются в использовании нейронных сетей глубокого обучения, что позволяет увеличить итоговую точность, однако, приводит к увеличению требований для вычислительной мощности.

Все нейронные методы можно разделить на два глобальных типа:

- Двухэтапные методы (основанные на регионах) - формируют набор гипотез, которые потом классифицируются и уточняются границы
- Одноэтапные методы - предполагают формирование набора прямоугольников при проходе нейронной сети

Region Based Convolutional Neural Networks (R-CNN)

TUSUR UNIVERSITY

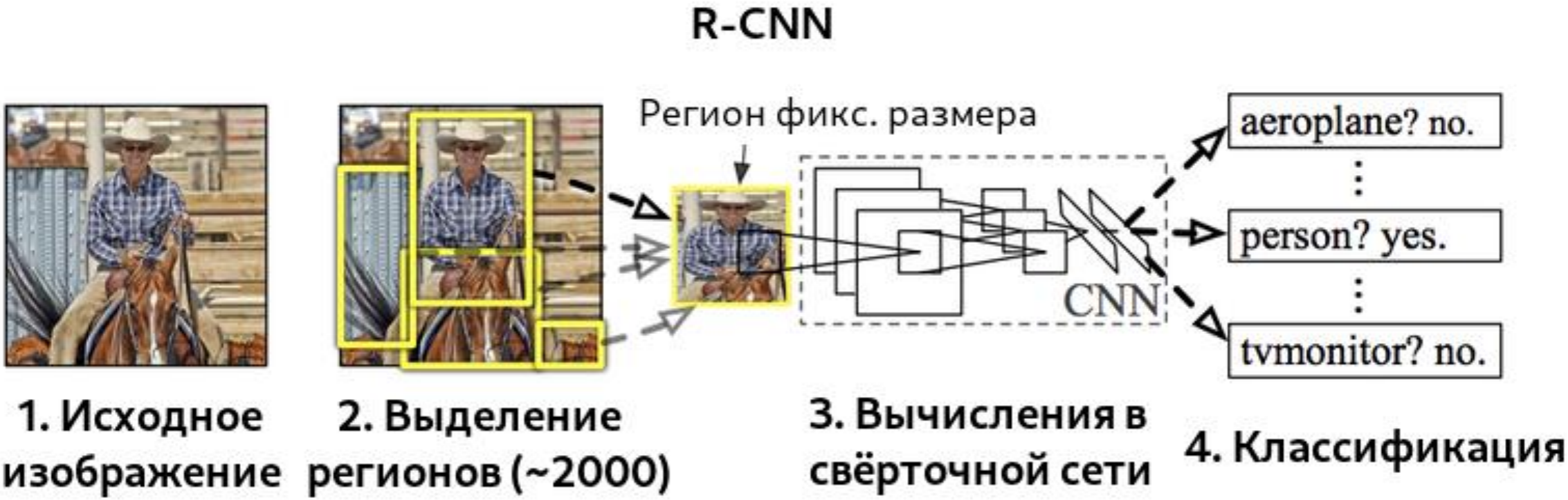
R-CNN — двухэтапный алгоритм, основанный на свёрточных нейронных сетях.

Алгоритм:

1. Первый этап
    1. Выделить регионы, которые содержат объекты
    2. Произвести свёртку выделенных регионов, для получения их признаков
2. Второй этап
    1. Произвести классификацию выделенных в регионах объектов, на основе полученных признаков с помощью классификатора SVM.

Достоинством данного метода является высокая скорость нахождения объекта, вне зависимости от его исходных размеров.

R-CNN



https://github.com/Meituan-Dianping/vision-ml/blob/master/rcnn_train.py

Fast R-CNN

ТУСУР
TUSUR UNIVERSITY

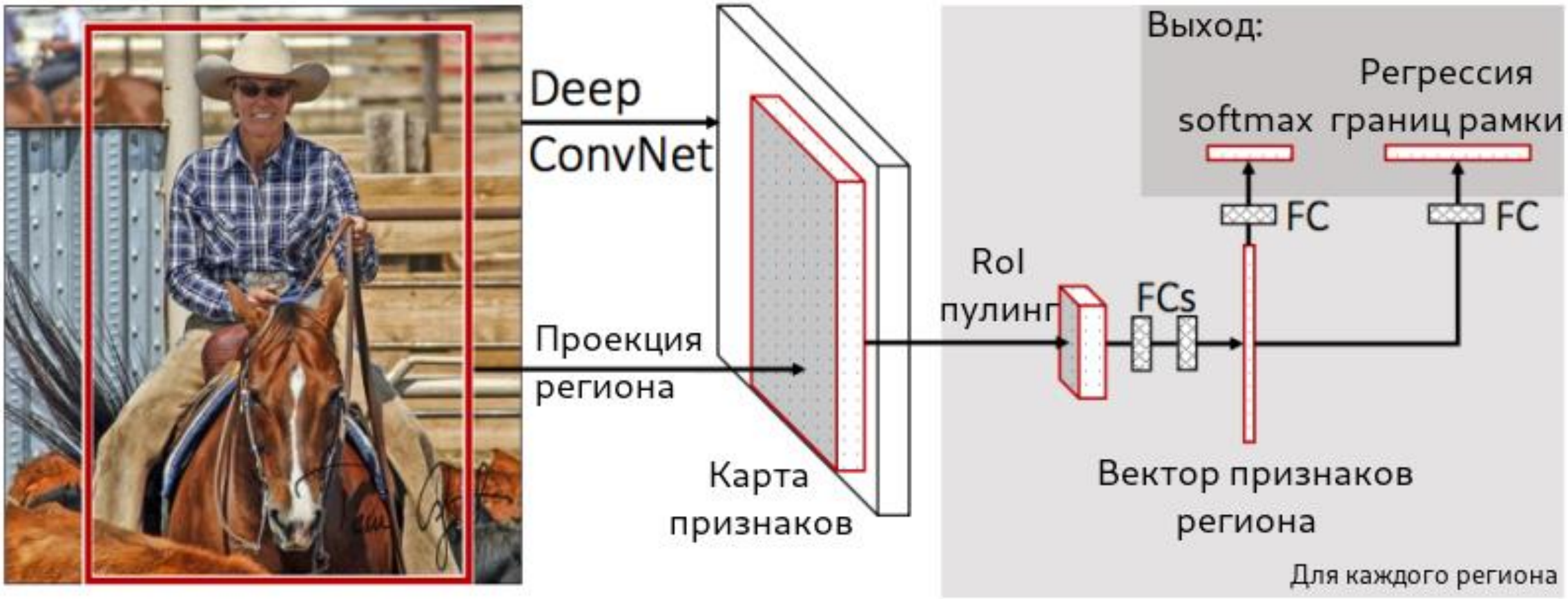
Характерной особенностью Fast R-CNN является подача на вход свёрточных слоёв полного изображения для получения общей карты признаков. Далее предложенные регионы накладываются на общую карту признаков, что приводит к уменьшению количества свёрток.

Поскольку регионы имеют разные размеры, перед классификацией необходимо привести полученные признаки к необходимым размерам с помощью операции RoIPooling – разделение региона на сетку, размерность ячеек которой совпадает с размерностью выхода, и выбор максимального значения в ячейках.

Далее полученные регионы отправляются в классификатор для определения объекта.

Достоинством данного метода является уменьшение времени обработки изображения до нескольких секунд.

Fast R-CNN

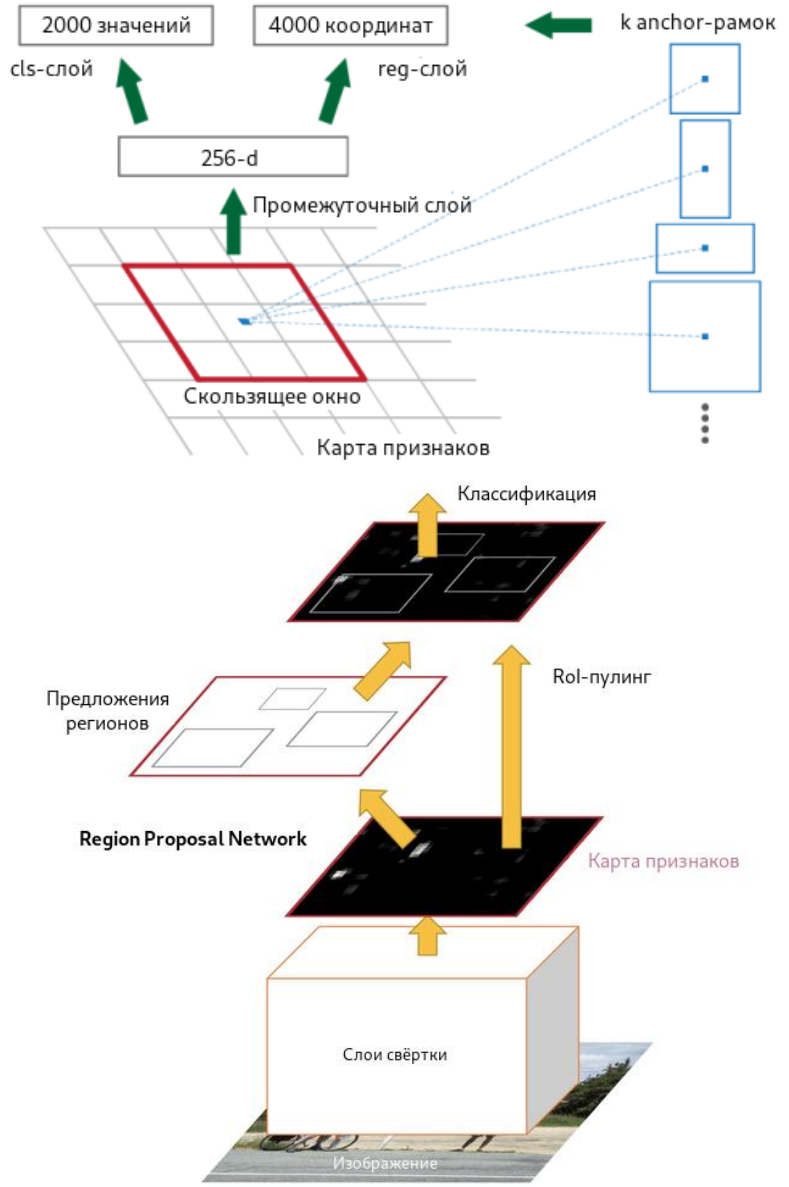

Deep
ConvNet
Проекция
региона
Карта
признаков
RoI
пулинг
FCs
Вектор признаков
региона
Выход:
softmax
Регрессия
границ рамки
FC
FC
Для каждого региона

Faster R-CNN

Fast R-CNN, как и оригинальный алгоритм R-CNN, использует для нахождения регионов селективный поиск, который выполняет предложения регионов, что является узким местом в производительности Fast R-CNN. Авторы алгоритма Faster R-CNN, призванного решить эту проблему, предложили вычислять регионы с помощью отдельного модуля Region Proposal Network (RPN) - свёрточная сеть, выполняющая роль генератора регионов по признакам исходного изображения.

Также был внедрён метод ключевых рамок, отвечающих за оптимизацию выбора регионов, благодаря прохождению скользящего окна в одном из трёх масштабов.

Использование данных методов, позволило перевести процесс детектирования в реальное время и увеличить точность распознавания, за счёт обучения RPN вместе с основной моделью.



https://github.com/you359/Keras-FasterRCNN

Mask R-CNN

ТУСУР
TUSUR UNIVERSITY

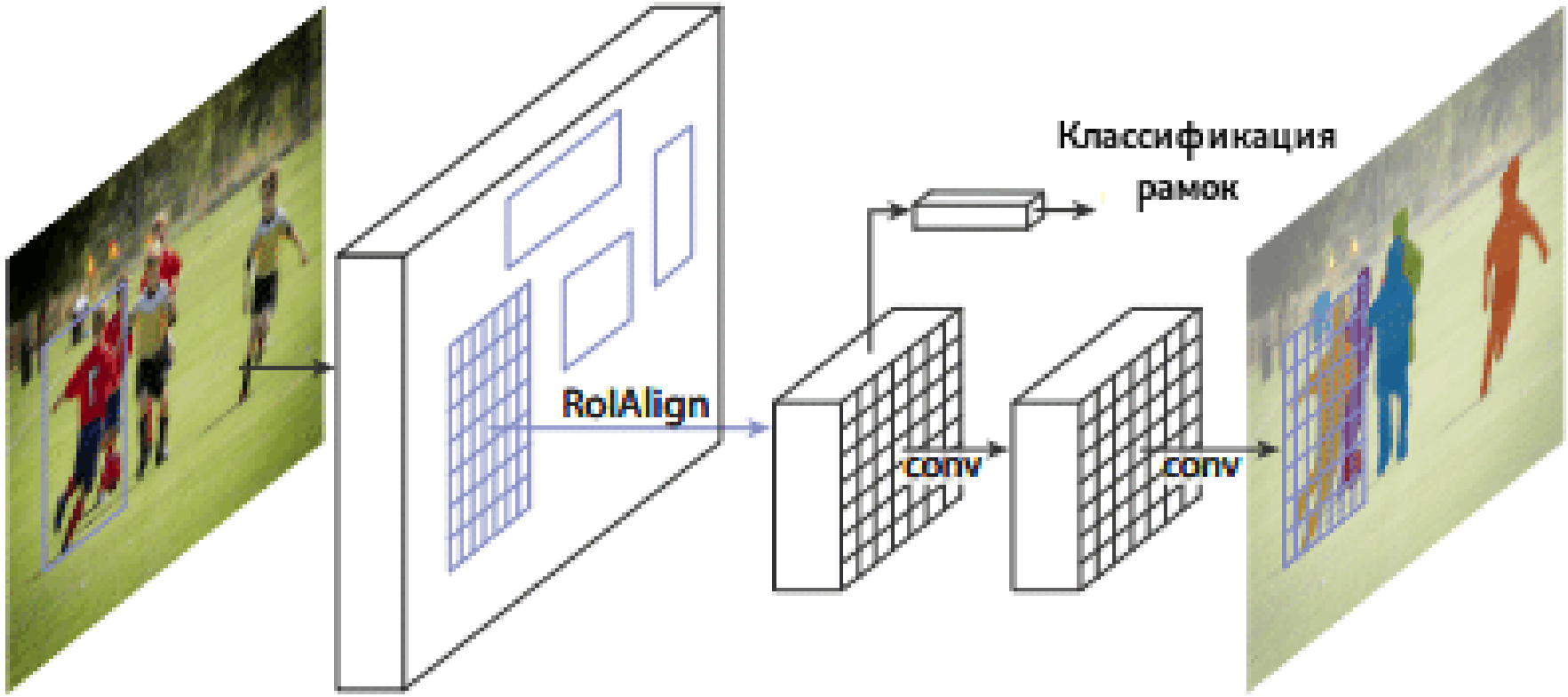
Mask R-CNN развивает архитектуру Faster R-CNN путём добавления ещё одной ветки, которая предсказывает положение маски, покрывающей найденный объект, и, таким образом решает уже задачу сегментации экземпляров.

Маска представляет собой прямоугольную матрицу, в которой 1 на некоторой позиции означает принадлежность соответствующего пикселя объекту заданного класса, 0 — что пиксель объекту не принадлежит.

Выделение маски происходит в class-agnostic стиле: маски предсказываются отдельно для каждого класса, без предварительного знания, что изображено в регионе, после чего классификатор выбирает наиболее близкую маску.

Одна из основных модификаций, возникших из-за необходимости предсказывать маску — изменение процедуры RoIPool(вычисляющей матрицу признаков для региона-кандидата) на так называемую RoIAlign, разницей которого является замена округления значений до целых на билинейную интерполяцию по четырём ближайшим точкам.

Mask R-CNN



https://github.com/matterport/Mask_RCNN/blob/master/mrcnn/model.py

You Only Look Once (YOLO)

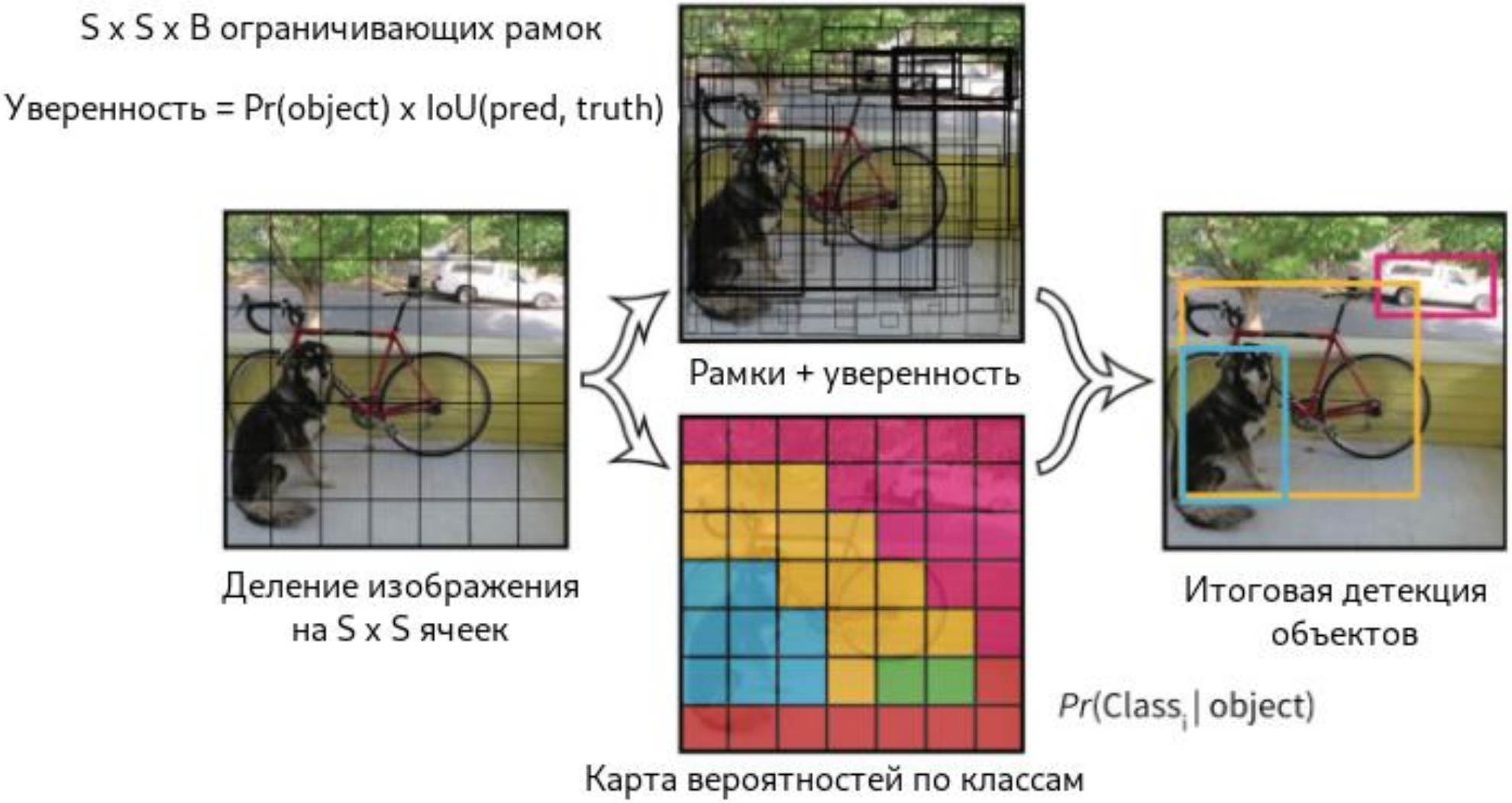
ТУСУР
TUSUR UNIVERSITY

Метод YOLO отличается от своих предшественников, тем что анализирует всё изображение целиком, без выделения регионов с предполагаемым расположением объектов. Это позволило производить детектирование объектов в реальном времени.

Алгоритм работы YOLO:

- Исходное изображение разбивается на сетку размером NxN;
- Проверка на наличие в ячейке центра объекта – если центр есть, то ячейка отвечает за найденный объект;
- Получение ограничивающих рамок относительно ячеек для каждого из объектов;
- Определить вероятность нахождения объекта в ячейках;
- На основе полученных параметров об ограничивающих рамках и вероятности нахождения объекта, формируются финальные результаты детектирования.

You Only Look Once (YOLO)



https://github.com/ultralytics/yolov5/blob/master/models/yolo.py

Single Shot MultiBox Detector (SSD)

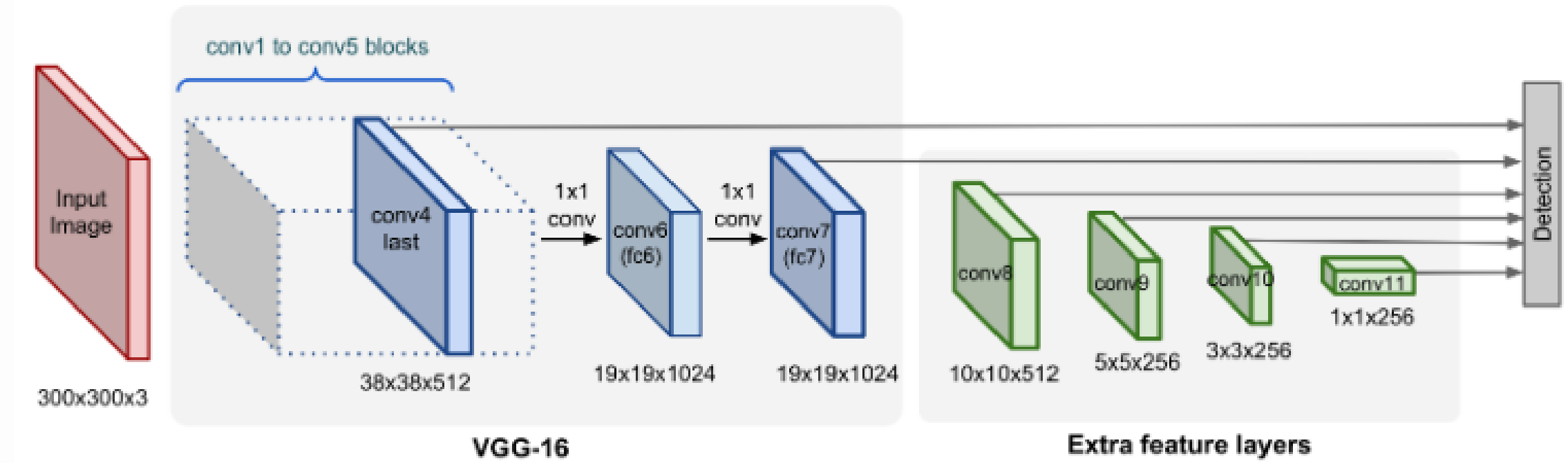
SSD использует идею использования пирамидальной иерархии выходов свёрточной сети для эффективного обнаружения объектов различных размеров.

Особенности:

- Изображение последовательно передаётся в свёрточные слои, уменьшающихся в размерах
- Каждый из свёрточных слоёв напрямую участвует в принятии финального решения модели
- За счёт сильной корреляции первых слоёв с ограничивающими рамками малых объектов, и последних слоёв с рамками больших объектов, SSD отлично справляется с обнаружением объектов различных масштабов
- Ввод предсказывания смещения ключевых рамок, которые масштабируются, что каждый слой отвечает за объекты определённого масштаба

Single Shot MultiBox Detector (SSD)

TUSUR UNIVERSITY



https://github.com/balancap/SSD-Tensorflow/blob/master/nets/vgg.py

TYCYP
TUSUR UNIVERSITY